Student Guide — Mandatory 2

Microsoft Defender XDR Incidents API

This guide explains how to connect to and use the Incidents API for Mandatory 2.
The API is a **simulation of the official Microsoft Defender XDR Incidents API** as operated by the fictional company **Øresund Industrial Manufacturing A/S**.

> **Reference:** Microsoft Defender XDR — List incidents
> The structure, field names, query parameters (`$top` , `$skip`), and OData response format used in this exercise follow the official Microsoft specification linked above.

Quick Facts

Property	Value
Base URL	`http://164.92.167.24`
Total incidents	205 (fixed — no new ones are added)
Token lifetime	48 hours
Max incidents per request	100
Rate limit	50 requests / minute, 1 500 requests / hour

Step 1 — Get Your Token

You need a token before you can call any incident endpoint. Use your school email address.

Request

```
POST /api/auth/token
Content-Type: application/json

{"email": "your.name@school.dk"}
```

Response

```
{
  "message": "Token retrieved successfully",
  "email": "your.name@school.dk",
  "token": "student-xxxxxxxxxxxxxxx",
  "expires_at": "2026-03-29T10:00:00+00:00",
  "expires_in_hours": 48.0,
  "instructions": "Use this token in Authorization header: Bearer <token>"
}
```

> **Token management is part of the assignment.** Your program must handle the token without manual intervention — either save it to a local file and reload it on the next run (checking expiry), or simply request a fresh token at the start of every run. Both approaches are acceptable.

Step 2 — Authenticate Requests

Include your token in the `Authorization` header on every request:

```
Authorization: Bearer student-xxxxxxxxxxxxxxx
```

Endpoints

GET /api/incidents — List incidents

Returns a page of incidents. Use `$top` and `$skip` for pagination.

Query parameter	Description	Default	Max
`$top`	How many incidents to return	10	100
`$skip`	How many incidents to skip	0	—
`severity`	Filter by severity	—	—
`status`	Filter by status	—	—
`threat_actor`	Filter by threat actor name	—	—

Severity values: `Low` , `Medium` , `High` , `Critical`
Status values: `Active` , `InProgress` , `Resolved`

The API holds 205 incidents in total. Use `$top=100` and increment `$skip` by 100 to page through the full dataset — with 205 incidents this requires **three pages** (`$skip=0` , `$skip=100` , `$skip=200`). Check `@odata.nextLink` in the response to determine whether a further page exists; when it is absent you have reached the last page.

Response structure

```
{
  "@odata.context": "https://api.security.microsoft.com/api/$metadata#Incidents",
  "@odata.count": 205,
  "value": [ ... ],
  "@odata.nextLink": "/api/incidents?$skip=100&$top=100",
  "pagination": {
    "skip": 0,
    "top": 100,
    "returned": 100,
    "filtered_total": 205,
    "unfiltered_total": 205
  },
  "api_version": "4.0",
  "mode": "mandatory2"
}
```

GET /api/incidents/{incidentId} — Single incident

Returns the full incident object including all alerts and entities.

GET /api/incidents/summary — Statistics

Returns aggregate counts — useful for reporting without fetching every incident.

Response


```
{
  "total_incidents": 205,
  "by_severity":      {"Critical": 46, "High": 74, "Medium": 64, "Low": 21},
  "by_status":        {"Active": 80, "InProgress": 65, "Resolved": 55},
  "by_classification":{"TruePositive": 150, "FalsePositive": 30,
"InformationalExpectedActivity": 20},
  "top_threat_actors":{"APT28": 28, "Lazarus Group": 24, ...},
  "top_malware_families": {"Emotet": 25, "Cobalt Strike": 22, ...},
  "recent_incidents": 34
}
```

Data Structure

Incident fields

```
incidentId       – unique ID, e.g. "INC1042"
incidentName     – e.g. "APT28 - Phishing Campaign"
createdTime      – ISO 8601 timestamp
lastUpdateTime   – ISO 8601 timestamp
status           – Active | InProgress | Resolved
severity         – Low | Medium | High | Critical
classification   – TruePositive | FalsePositive | InformationalExpectedActivity
determination    – Malware | Phishing | MultiStagedAttack | ...
assignedTo       – email or null
tags             – [ "APT28", "Emotet", "Øresund Industrial" ]
threatFamily     – malware family name
mitreTechniques  – [ "T1566.001", ... ]
impactedEntities – { "users": 3, "machines": 2, "mailboxes": 1 }
comments         – [ { "comment": "...", "createdBy": "...", "createdTime": "..." } ]
summary          – text description
alerts           – array (see below)
```

Alert fields

```
alertId          – e.g. "ALT1042-0"
title            – description of what was detected
category         – Malware | Phishing | Lateral Movement | Exfiltration | C2 | UnauthorizedAccess
severity         – Low | Medium | High | Critical
detectionSource  – which Microsoft product detected it
machineId        – host identifier, e.g. "ERP01-TCORP"
computerDnsName  – FQDN of the affected host
firstActivity    – ISO 8601 — first time this activity was observed
lastSeen         – ISO 8601 — last time this activity was observed
entities         – object containing IOCs (see below)
```

Entities (IOCs) inside each alert

```json
"entities": {
  "ips":         ["192.168.20.25", "91.108.56.175"],
  "domains":     ["command-control-eu.net"],
  "fileHashes": ["d41d8cd98f00b204e9800998ecf8427e"],
  "emails":      ["sysadmin.nielsen@oresund-industrial.dk"],
  "processes":  ["powershell.exe", "cmd.exe"]
}
```

Suggested SQLite Schema

The assignment requires you to store alert data in a flat SQLite table.
Below is a minimal schema that covers the required columns.

```sql
CREATE TABLE IF NOT EXISTS alerts (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    alertID         TEXT,
    incidentId      TEXT,
    category        TEXT,
    machineID       TEXT,
    firstSeen       TEXT,
    timestamp       TEXT,
    detectionSource TEXT
);
```

> `id` is an auto-generated integer primary key — this avoids conflicts if `alertID` values are not unique.
> `timestamp` should store the date and time your program inserted the row — you are responsible for generating and passing this value when inserting data.
> These are the **minimum** required columns. You are free to add more.

Inserting data — always use parameterised queries

> **Always use `?` placeholders — never string concatenation.**
> Some alert values in this dataset contain special characters that will break your database if you build SQL strings with f-strings or `+`.

Error Handling

The assignment requires error handling on both the **network requests** and the **database operations**.

For requests: catch timeouts, HTTP errors (`4xx` / `5xx`), and general connection errors using the `requests` exceptions (`requests.exceptions.Timeout` , `HTTPError` , `RequestException`).

For the database: wrap inserts in a `try/except` and call `conn.rollback()` on failure to avoid corruption.

Recommended Modules

Module	Purpose	Link
`requests`	HTTP calls to the API	https://docs.python-requests.org
`json`	Parse JSON responses (often not needed — `response.json()` handles it)	https://docs.python.org/3/library/json.html
`sqlite3`	SQLite database	https://docs.python.org/3/library/sqlite3.html
`mysql.connector`	MySQL / live DB (optional)	https://dev.mysql.com/doc/connector-python/en/connector-python-example-connecting.html

Install `requests` if not already present: `pip install requests`

Rate Limits

Limit	Value
Per minute	50 requests
Per hour	1 500 requests
Max incidents per request	100

If you exceed a limit the API returns `401`. Add `time.sleep(1.5)` between paginated calls to stay safe.

Error Reference

HTTP Status	Meaning
200	OK
401	Invalid / expired token, or rate limit exceeded
404	Incident ID not found
403	Access denied (admin-only endpoint)

> If you get `401` and your token is correct, you may have hit the rate limit. Wait 60 seconds and retry.

Example: Python Script

Here's a simple Python example to get you started:

```python
import requests

# Configuration
API_BASE = "http://164.92.167.24"
EMAIL = "your.name@school.dk"

# Get token
def get_token():
    response = requests.post(f"{API_BASE}/api/auth/token", json={"email": EMAIL})
    return response.json()["token"]

# Get incidents
def get_incidents(token):
    headers = {"Authorization": f"Bearer {token}"}
    response = requests.get(f"{API_BASE}/api/incidents", headers=headers)
    return response.json()

token = get_token()
incidents = get_incidents(token)
print(incidents["value"])
```